Fundamentals of API Development

Breakout Lab 2.2: Implementing Basic API Security Rules

Day 3

KavinSchool

Lab Overview

Goal: Implement a request gate that checks authentication, authorization, and rate limiting, in that order, against a set of pre-configured requests, and understand why the order matters.

- **Format:** Individual or pair work
- **Tools:** IntelliJ, the companion repo (API does not need to be running)

> **NOTE**
>
> You just watched `DemoJwtAuthNGate` gate four requests on authentication and `DemoTokenBucketRateLimiter` show a token bucket reject bursts. This lab asks you to wire all three checks together in one gate.

What's in the Companion Repo

Open `day3/lab2_2_implementing_basic_api_security_rules/` in the companion repo.

Path	What it is
`start/SecurityGate.java`	The gate you complete. `checkAuthentication` is the worked example; two TODOs are yours.
`solution/SecurityGate.java`	Completed gate with two additional requests exercising edge cases.

NO API NEEDED

This lab is pure Java, no Spring context, no running server. The requests are pre-configured objects with tokens, roles, and rate-limit counters.

Step 1: Read the Worked Example

Authentication Is Done

Open `start/SecurityGate.java`. Read `checkAuthentication` first, it is the worked example:

- Receives an `IncomingRequest` with a token
- Validates the token is present and not expired
- Returns `true` if authenticated, `false` with `401` otherwise

Run it now:

```
mvn exec:java -pl day3/lab2_2_implementing_basic_api_security_rules/start \
  -Dexec.mainClass="com.kavinschool.api.SecurityGate"
```

Requests `R1` through `R5` exercise valid, expired, tampered, and missing tokens. Only `R1` passes authentication.

Step 2: Implement Authorization and Rate Limiting

TODO 1: Check Authorization

Implement `checkAuthorization`. The gate already calls it; you fill in the logic.

- Receive an `IncomingRequest` with a `role` and a `requiredRole`
- If the request's role does not match the required role, return `false` with `403`
- If it matches, return `true`

> **ORDER MATTERS**
>
> Authorization only runs after authentication passes. A request with no token should fail at auth (401), never reach authZ (403).

TODO 2: Check Rate Limiting

Implement `checkRateLimit`. The gate already calls it; you fill in the logic.

- Receive an `IncomingRequest` with a `remainingTokens` count
- If remaining tokens is zero or less, return `false` with `429`
- If tokens remain, decrement the count and return `true`

THINK ABOUT ORDER

What would go wrong if `checkRateLimit` ran before `checkAuthentication`? An unauthenticated attacker could drain the rate-limit bucket just by sending requests, a DoS vector that costs nothing.

The Request Gauntlet

Your finished gate should process every request through the same three checks, in this order:

Order	Check	Rejection code
1	Authentication, who are you?	`401`
2	Authorization, what can you do?	`403`
3	Rate limiting, how often?	`429`

STRETCH

Add a fourth check, `checkAudienceClaim`, that rejects a request whose token wasn't issued for this specific API. Add an `audience` field to `IncomingRequest` to support it, and decide where in the order it belongs.

Lab Completion Checklist

Checklist

- [ ] Worked example (`checkAuthentication`) read and understood
- [ ] `checkAuthorization` implemented and rejecting wrong roles with `403`
- [ ] `checkRateLimit` implemented and rejecting exhausted tokens with `429`
- [ ] All three checks run in the correct order: auth → authZ → rate limit
- [ ] At least one request exercises each rejection path (401, 403, 429)

DONE?

Compare your gate's output with a neighbor. Run the solution's additional requests (`R6` , `R7`) and see which check stops each one, the correct answer is a discussion, not a single number.

Lab 2.2 Complete

Day 4's test suite will need test cases for all three gates

KavinSchool